

1.0 面向对象高级



1

面向对象高级

- ◆ 封装
- ◆ 继承
- ◆ 多态
- ◆ 案例

1.1 封装

封装

- 封装就是把数据(属性)和操作数据的函数(方法)捆绑在一起, 形成一个独立的单元(类), 并隐藏内部的实现细节, 只对外暴露必要的功能(方法)。

公共属性、公共方法

```
class Car:
    # 类属性 (所有实例对象共享的)
    wheel = 4 # 轮胎数量
    tax_rate = 0.1 # 购置税税率

    def __init__(self, c_color, c_brand, c_name):
        # 实例属性
        self.color = c_color
        self.brand = c_brand
        self.name = c_name

    def start(self): # 启动
        print(f'{self.brand} {self.name} 正在启动...')

    def stop(self): # 停止
        print(f'{self.brand} {self.name} 停止行驶...')
```

私有属性、私有方法

```
class Car:
    # 类属性 (所有实例对象共享的)
    wheel = 4 # 轮胎数量
    tax_rate = 0.1 # 购置税税率

    def __init__(self, c_color, c_brand, c_name, c_owner):
        # 实例属性
        self.color = c_color
        self.brand = c_brand
        self.name = c_name
        self.__owner = c_owner # 私有属性

    def start(self): # 启动
        print(f'{self.brand} {self.name} 正在启动...')

    def stop(self): # 停止
        print(f'{self.brand} {self.name} 停止行驶...')

    def __control_fuel(self): # 私有方法
        print(f'{self.brand} {self.name} 控制燃油分配...')
```

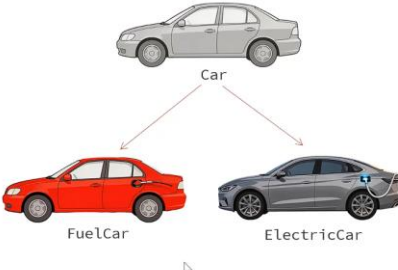
私有: 私有的属性和方法只能在类的内部使用; Python中并没有真正的私有机制, 约定在私有属性名和方法名前加__ (两个下划线)。

py 中并无真正的私有, 只是约定;

py 内部将 私有属性/方法前加上"_" + 类名, 若在访问的私有前加上, 则可以直接访问;

1.2 继承

- 继承描述的是两个类之间的关系, 子类继承父类, 就可以获取到父类的属性和方法。(非私有)



```
class Car:
    def __init__(self, c_color, c_brand, c_model):
        self.color = c_color
        self.brand = c_brand
        self.model = c_model

    def start(self): # 启动
        print(f'{self.brand} {self.model} 正在启动...')

    def stop(self): # 停止
        print(f'{self.brand} {self.model} 停止行驶...')

    def run(self): # 行驶
        print(f'{self.brand} {self.model} 正在行驶...')
```

父类

```
class FuelCar(Car):
    pass
```

指定父类的类名 子类

```
class ElectricCar(Car):
    pass
```

指定父类的类名 子类

私有方法/属性不能继承;

继承-重写

- **重写**是指子类继承父类后，如果父类中的方法不满足需求，可以在子类中重新定义父类中已有的方法（方法名相同），从而用子类的实现替换父类的实现。

```
class Car:
    def __init__(self, c_color, c_brand, c_name):
        self.color = c_color
        self.brand = c_brand
        self.name = c_name

    def start(self): # 启动
        print(f'{self.brand} {self.name} 正在启动...')
    def stop(self): # 停止
        print(f'{self.brand} {self.name} 停止行驶...')
    def charge(self):
        print(f'{self.brand} {self.model} 正在补充燃料...')

class FuelCar(Car): # 油车类
    def charge(self): # 补充燃料
        print(f'{self.brand} {self.model} 正在加油 ...')
```

```
class Car:
    def __init__(self, c_color, c_brand, c_name):
        self.color = c_color
        self.brand = c_brand
        self.name = c_name

    def start(self): # 启动
        print(f'{self.brand} {self.name} 正在启动...')
    def stop(self): # 停止
        print(f'{self.brand} {self.name} 停止行驶...')
    def charge(self):
        print(f'{self.brand} {self.model} 正在补充燃料...')

class FuelCar(Car): # 油车类
    def charge(self): # 补充燃料
        super().charge()
        print(f'{self.brand} {self.model} 正在加油 ...')
```

注意：如果子类在重写父类的方法时，需要调用父类的方法，可以通过 父类名.方法名(self) / super().方法名() 方式来调用。

多继承

- 多继承指的是一个子类，同时继承了多个父类的情况（会将多个父类中的非私有的属性和方法都继承下来）。
- 语法：

```
class 子类名(父类名1, 父类名2, 父类名3, ...):
    代码
    ...
```

MRO : Method Resolution Order (方法解析顺序)

注意：当一个类继承了多个父类时，默认优先使用第一个父类中的同名属性或方法，可以使用 类名.__mro__ 属性 或 类名.mro() 方法查看调用顺序。

MRO 顺序:先访问自己的构造函数,若无->第一个父类,若无->第二个父类,若无->第一个父类的父类

```
临时文件和控制台
46 print(WenJieCar.__mro__)
47 print(WenJieCar.mro())

运行 04.面向对象高级-继承(多继承) x
D:\develop\Python\python.exe "D:\Python-Project\py_project01\第6章\04.面向对象高级-继承(多继承).py"
(<class '__main__.WenJieCar'>, <class '__main__.Car'>, <class '__main__.HuaweiAiDriving'>, <class 'object'>)
[<class '__main__.WenJieCar'>, <class '__main__.Car'>, <class '__main__.HuaweiAiDriving'>, <class 'object'>]

# 问界汽车
class WenJieCar(Car, HuaweiAiDriving): 1个用法
    def __init__(self, brand, model, color, owner, version = "V1.0"):
        super().__init__(brand, model, color, owner)
        HuaweiAiDriving.__init__(self, version)

# MRO: Method Resolution Order --> 方法解析顺序
if __name__ == '__main__':
    c = WenJieCar(brand="BMW", model="X5", color="黑色", owner="张三", version="V1.1")
    print(c.__dict__)

    # print(WenJieCar.__mro__)
    # print(WenJieCar.mro())
```

故想要所有父类的构造方法,则自己写构造函数即可;

1.3 多态

多态

- 多态是指同一个方法,具有不同的形态、行为、表现。

```
class Car:
    ...
    def charge(self):
        print('正在补充燃料...')

class FuelCar(Car):
    def charge(self):
        print('燃油汽车正在加油...')

class ElectricCar(Car):
    def charge(self):
        print('电动汽车正在充电...')
```

```
def handle_charge(car: Car):
    car.charge()
    # 参数类型为父类,支持传入任意子类

handle_charge(FuelCar("BMW", "X5", "白色", 500, "张三"))
handle_charge(ElectricCar("BYD", "汉", "白色", 500, "张三"))
# 同一调用语句,传入对象不同,触发不同行为
```

字幕已关闭

类型注解并不强制,只是说明;

多态-鸭子类型

- 鸭子类型(Duck Typing)是指如果它走路像鸭子,叫起来叫鸭子,那么它就是鸭子。
- 在鸭子类型中,我们关注的是对象的行为(它有什么方法),而不是对象的类型(它是什么类)。

```
class Dog:
    ...
    def swimming(self):
        print(f'{self.age} 岁的 {self.name} 正在游泳...')

class Duck:
    ...
    def swimming(self):
        print(f'{self.age} 岁的 {self.name} 正在游泳...')

class Pig:
    ...
    def swimming(self):
        print(f'{self.age} 岁的 {self.name} 正在游泳...')
```

```
def go_swimming(duck):
    duck.swimming()

# 测试代码
if __name__ == '__main__':
    go_swimming(Dog("旺财", 4))
    go_swimming(Duck("唐老鸭", 2))
    go_swimming(Pig("佩奇", 1))
```

提示: 鸭子类型的优势是不需要存在继承关系,只要对象有相应的方法就能使用。

py 中的多态并不依赖于继承;只要传入的参数能够执行此函数,则可以传入;

2.0 FastAPI 基础

Web初识

- 前端程序：负责前端页面的展示，网页由 HTML(负责网页的结构)、CSS(负责网页的表现)、JavaScript(负责网页的动作) 三个部分组成。
- 服务端程序：可以基于Python中的Django、Flask 或 FastAPI来进行开发。

```
graph LR; Frontend[前端: 界面展示] --> FastAPI[FastAPI]; FastAPI --> Files[文件: 数据存储和管理]; Files --> FastAPI; FastAPI --> Frontend
```

FastAPI

- FastAPI是一个现代、快速、高性能的Web框架，用于基于标准的Python类型提示构建API接口服务。
- 官网：<https://fastapi.org.cn>

```
graph LR; Client[Client] -- HTTP请求 --> Server[Server]; Server -- HTTP响应 --> Client
```

API接口：应用程序编程接口(Application Programming Interface)，就是对外提供的功能入口，供别人来调用（比如：[天气查询的API接口](#)）。

基于 fastapi 来开发 api 接口服务;

FastAPI入门

- 使用步骤：
 1. 导入FastAPI
 2. 创建FastAPI实例对象
 3. 创建路径操作函数，定义访问路径
 4. 运行FastAPI服务

```
fastapi dev "xxx.py"
uvicorn xxx:app --reload
```

```
from fastapi import FastAPI

# 创建FastAPI实例
app = FastAPI()

# 定义功能接口
@app.get("/")
def root():
    return {"message": "Hello World"}

# 定义功能接口
@app.get("/users")
def get_users():
    return [{"id": 1, "name": "Alice", "age": 18}, {"id": 2, "name": "Bob", "age": 20}]

# 启动FastAPI服务器
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

启动方式:1>fastapi dev "***.py":先执行 pip install "fastapi[standard]" 后才能使用;

2>uvicorn fastapi_test:app --reload 直接启动

3>代码中用 uvicorn web 服务器启动;

2.1 Restful api 接口规范

Restful

- Restful指的是遵循REST架构风格的API接口服务，而REST (REpresentational State Transfer)，表述性状态转换，它是一种软件架构风格。

传统风格url	请求方式	含义	备注
http://localhost:8000/user/getById?id=1	GET	查询id为1的用户	不规范 难维护
http://localhost:8000/user/saveUser	POST	新增用户	
http://localhost:8000/user/updateUser	POST	修改用户	
http://localhost:8000/user/deleteUser?id=1	GET	删除id为1的用户	

REST风格url	请求方式	含义	备注
http://localhost:8000/users/1	GET	查询id为1的用户	URL定位资源 HTTP动词描述操作 简洁、规范、优雅
http://localhost:8000/users/1	DELETE	删除id为1的用户	
http://localhost:8000/users	POST	新增用户	
http://localhost:8000/users	PUT	修改用户	

增删改查是相对于前端来讲;
给定义路径,请求方式的规范;

注意: REST是风格,是约定方式,约定不是规定,可以打破。

注意: 描述功能模块通常使用复数形式(加s),表示此类资源,而非单个资源。如: users、books、items。

3.0 实战说明

web 应用常识:

1. 浏览器渲染前端,交互前端->前端向后端发送路径,请求->后端处理,返回数据->前端渲染

基础环境搭建

- 创建项目文件夹,将资料中的 static 目录拷贝到项目中。
- 编写python程序,定义路径操作函数,访问前端HTML页面。

```
main.py
from fastapi import FastAPI
from fastapi.responses import FileResponse

# 创建FastAPI实例
app = FastAPI(title="汉字谜语")

# 挂载静态文件目录
app.mount("/static", StaticFiles(directory="static"), name="static")

# 定义路径操作函数
@app.get("/")
def root():
    return FileResponse("static/index.html")
```

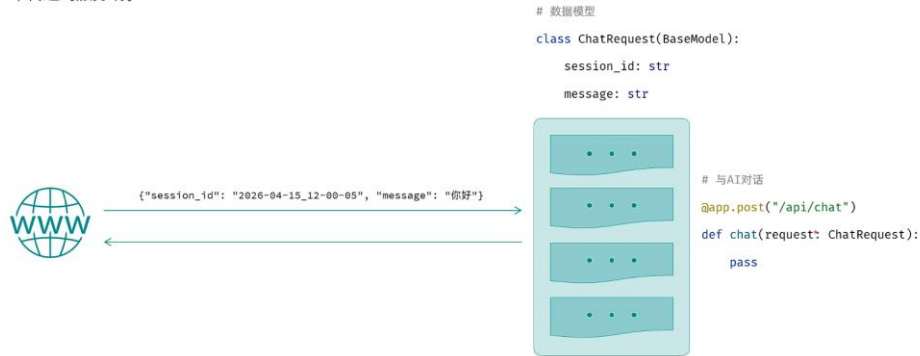
任何以/static开头的路径都会交给这儿处理

FastAPI内部使用的名称(可以自由定义)

HTML、CSS、JS等静态资源存放目录

与AI交互

- 在与AI进行交互式，前端传递给服务端的参数包含两项，分别为session_id 与 message，并以json格式在请求体中传递到服务端。



字幕已关闭

BaseModel: 是Pydantic库提供的父类 (FastAPI 深度集成了 Pydantic)，用于定义FastAPI数据模型和数据验证规则。

日志记录

- 为了能够灵活的控制项目中日志的输出，我们可以通过官方提供的logging模块来输出日志，具体做法如下：

```
from fastapi import FastAPI
import logging

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - [%(filename)s:%(lineno)d] - %(message)s"
)

# 创建FastAPI实例
app = FastAPI()

# 定义路径操作函数
@app.get("/")
def root():
    logging.info("访问首页~")
    return FileResponse("static/index.html")
```

字幕已关闭

日志级别: 日志级别就是给日志信息贴上的“重要性标签”，常见的级别有：DEBUG、INFO、WARNING、ERROR、FATAL（日志级别依次升高）。

统一异常处理

- 项目中的功能较多，目前我们并未考虑异常处理，可以借助于FastAPI中的统一异常处理方案来处理异常。

```
# 统一处理异常信息
@app.exception_handler(Exception)
def handle_exception(request: Request, exc: Exception):
    logging.error(f"处理异常，请求路径: {request.url}, 异常信息: {exc}")
    return JSONResponse(content={"code": 500, "message": "服务器内部错误"})
```



Exception:接收所有异常;
参数:接收请求类型,异常类型;
返回给前端 json 提示信息;

完结撒花

学习路线

零基础8天速成

Python数据分析

Linux+MySQL+Numpy+Pandas

65.2万 1623 36:09:16

黑马程序员Python数据分析全套视频教程，一套搞定Linux、MySQL...

2025-10-23

7天掌握FastAPI框架

PythonWeb开发

AI大模型核心技能

60.5万 2284 14:55:30

黑马程序员PythonWeb开发：Fast API从入门到实战视频教程，涵...

2025-12-10

通关手册 全网最全 有手就会

Coze智能体

6小时搞定AI应用

69.0万 1799 05:28:32

黑马程序员全网最全Coze智能体入门到项目实战全套教程，从AI A...

2025-12-15

零基础玩转

Dify

5小时掌握 Prompt RAG 工作流

30.8万 643 04:23:41

黑马程序员零基础玩转Dify，5小时极速入门Agent开发，从Prompt...

03-16

2026最新版

LangChain全家桶

Agent+RAG 实战天花板

42.9万 1082 07:18:23

黑马程序员2026最新版LangChain+LangGraph开发实战全套视频...

03-20

LangChain最新版

大模型RAG

Agent智能体实战项目

189.3万 7229 15:31:31

黑马程序员大模型RAG与Agent智能体项目实战教程，基于主流的...

01-21